Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00. Fax: +46 (0)771 97 50 01.

Projektuppgift

Programmering i C#.NET, DT071G

Projekt

Alfons Eriksson Sveiven

Projekt
Alfons Eriksson Sveiven 2026-10-20

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Alfons Eriksson Sveiven, aler2507@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: 3, 2026

Projekt
Alfons Eriksson Sveiven 2026-10-20

Sammanfattning

Uppgiften är att tillämpa de kunskaper vi lärt oss under kursen för att skapa en mer omfattande och användbar applikation skriven i C#.

Vi fick använda oss utav fler tillägg än bara ramverket .NET. Vi ska skapa en projektrapport som beskriver arbetet med projektuppgiften och en enklare presentation av arbetet i videoformat.

Jag valde att göra en konsolapplikation med ett hänga gubbe spel. Källkoden versionshanterades med Git och den färdiga konsolapplikationen finns uppladdad till github.

Uppgiften går ut på att skapa en applikation i C# och använda ramverket .NET på ett valfritt sätt. Vi skulle försök hitta någon form av idé till applikation som drar nytta av de teorier och tekniker ni har studerat i kursen. Applikationen fick vara konsol-, desktop-, webb- eller mobilbaserad.

Min konsolapplikation är en fungerande och användarvänlig hänga gubbe där man kan spela. Det finns highscore, jag har använt mig av JSON, arrayer, filhantering och ASCII grafik.

1 Introduktion

Syftet vara att skapa en fungerande applikation för de C# .NET koncept vi lärt oss under kursens gång. Målet var att applikationen skulle vara lätt att förstå för användaren och välfungerande.

Uppgiften gick ut på att tillämpa de kunskaper vi lärt oss under kursen för att skapa en omfattande och användbar applikation skriven i C#. Vi fick använda oss av flera tillägg än bara ramverket .NET.

1.1 Problemställning

Jag valde att skapa en konsolapplikation för det klassiska spelet hänga gubbe. Spelet går ut på att användaren ska gissa ett hemligt ord, en bokstav i taget. Varje felaktig gissning resulterar i ett steg närmare att förlora. Spelaren har 9 liv och för varje korrekt gissning fylls en bokstav i ordet.

Problemen som skulle lösas var att välja ett slumpmässigt ord från en lista. Att visa ordet som understreck och fylla i korrekta bokstäver när rätt gissning skedde. Att hålla koll på felaktiga gissningar och antal kvarvarande liv. Att rita en ASCII-grafik för gubben som hängs. Att ge tydlig feedback och validering för användarens inmatning så att det inte går att skriva fel. Att spara statistiken mellan körningar.

1.2 Spela igen!

För att ge programmet en anledning att köras igen ville jag spara resultatet mellan varje omgång. Antalet vinster och förluster lagras i en fil så att statistiken finns kvar även efter att programmet stängs. Ännu en anledning att köra igen är att det finns flera ord. Om det är för få och man får samma igen finns det ingen anledning att någonsin starta det igen.

1.3 Tekniska krav

Applikationen fick vara konsol-, desktop-, webb- eller mobilbaserad. Jag valde konsol eftersom det är det vi har använt tidigare. Den behövde vara välkommenterad och använda sig av det vi gjort i tidigare uppgifter. Tex “random”, arrayer, loopar, felhantering av inmatning och JSON.

1.4 Avgränsningar

Pga tidsbrist och uppgift i kursen projektledning som pågick samtidigt behövdes en del avgränsningar. Bara det faktum att det är en konsolapplikation tillför också en del avgränsningar. Hela idén med hänga gubbe var inte min första men kändes mer görbart än andra ideér. Ordlistan är tex hårdkodad osv osv.

2 Teori

2.1 C# och .NET

C# är ett objektorienterat språk som kör på microsofts .NET-plattform. Det är plattformsoberoende och har brett stöd. Det kan köras på allt ifrån IoT-enheter till telefoner, datorer och servrar.[1]

C# .NET har flera program varianter, konsol, desktop, webb eller mobil, jag skrev uppgiften som en konsolapplikation. Allt är skrivet utan externa paket förutom system.text.json som tillåter JSON fungera i C# .NET

2.2 JSON

JSON, JavaScript Object Notation är ett textformat för att lagra och överföra data.[2]

Formatet används i moderna webbtjänster och i detta projekt användes det för att spara highscore. Klassen Scoredata serialiseras till filen hangman_score.json med:

```
        string json = JsonSerializer.Serialize(score, new
JsonSerializerOptions { WriteIndented = true });

        File.WriteAllText(scoreFile, json);
```

2.3 Random

Klassen System.Random används för att välja ett ord ur listan. Metoden Next(words.Count) ger ett heltal mellan 0 och antalet ord minus ett.

```
Random rnd = new Random();

        string secretWord = words[rnd.Next(words.Count)];
```

2.4 Array

char[] används för att visa ordet som gissas på med understreck som byts ut mot bokstäver.

List<char> används för att spara de felaktiga gissningarna och List<string> innehåller alla hemliga ord.

2.5 Filhantering

Klasserna File.Exists, File.ReadAllText och File.WriteAllText används för att läsa och skriva ut highscore filen. Om inte filen finns så skapas en ny.

2.6 ASCII-grafik

Det är den digitala motsvarigheten till skrivmaskinskonst. En enkel textbaserad konst som utnyttjar skrivtecken (bokstäver, siffror och specialtecken). [3]

Istället för att bara skriva “liv kvar = 4”. Visas en enkel ASCII grafik/konst i konsolen för användaren att se hur nära hen är att förlora. Figuren lagras i en sträng array med 10 steg och väljs med indexet lives.

2.7 Git

Git är ett versionshanteringssystem som gör det möjligt att spåra ändringar i källkoden över tid. Det används dels för att samarbeta, återgå till tidigare versioner och att hålla ordning på kodbasen. Koden för detta projekt versionshanteras med Git.[4]

3 Metod

Först planerade jag hur konsolapplikationen skulle se ut. Jag visste genom tidigare uppgifters kod hur saker och ting kunde genomföras vilket gjorde hela uppgiften enklare att handskas med, då jag kan ta den tidigare skrivna koden och anpassa efter det nya behovet. Men först gjorde jag en wireframe.

Denna visualiserade vad jag behövde göra för något innan programmeringen påbörjades. Som en titt tillbaka till tidigare uppgifter valde jag igen att använda mig utav === TITEL === för att behålla teemat. och resten säger sig själv. I detta läge var jag inte säker på hur jag skulle få figuren att hängas, ascii tillkom senare i processen.



Figur 1. Hänga gubbe.

Projekt
Alfons Eriksson Sveiven 2026-10-20

Applikationen behövde fem huvudsakliga delar. Den första var ordlistan och ett sätt att kalla på den vid spelets start. Den andra var ord gissningen, alltså att fylla i vad som gissas rätt på. Ett sätt att dra den klassiska grafiken var den tredje. Det fjärde var att visa vad som gissats på och det femte var den interaktiva delen. Att navigera applikationen, att gissa bokstäver med felhantering om man gissar samma igen etc.

De verktyg som används var Visual studio Code med dotnet CLI .NET 10 med inbyggda biblioteken System, System.Collections.Generic, System.IO och System.Text.Json. Ingen extern NuGet-paket utöver det som redan ingår i .NET behövdes.

4 Konstruktion

Programmet består av en Program klass med statistiska metoder och en hjälpklass ScoreData. Huvudloopen, alltså det man ser när man öppnar applikationen ser ut såhär:

```
while (true)
        {
            Console.Clear();
            ShowMenu();

            Console.Write("Välj: ");
            string choice =
Console.ReadLine()?.Trim().ToUpper();

            if (choice == "1")
            {
                PlayGame();
            }
            else if (choice == "2")
            {
                ShowHighScore();
            }
            else if (choice == "X")
            {
                Console.WriteLine("Hej då!");
                break;
            }
            else
            {
                Console.WriteLine("Ogiltigt val.
Tryck Enter...");
                Console.ReadLine();
            }
        }
```

När man startar PlayGame() sparas ett ord från listan i secretWord. Ordet man gissar på visas genom char[] som initialiseras med bara understräck:

```
        char[] guessed = new char[secretWord.Length];
        for (int i = 0; i < guessed.Length; i++)
            guessed[i] = '_';
```

För varje gissning kollar programmet om bokstaven finns i ordet. Om den gör det uppdateras guessed, annars minskas lives och bokstaven läggs till i listan över felaktiga gissningar.

```
bool found = false;
            for (int i = 0; i < secretWord.Length;
i++)
            {
                if (secretWord[i] == guess)
                {
                    guessed[i] = guess;
                    found = true;
                }
            }

            if (!found)
            {
                wrongGuesses.Add(guess);
                lives--;
            }
```

Figuren ritas ut med metoden DrawHangman(int lives) som innerhåller en array med 10 ascii-strings. Indexet motsvarar antal liv användaren har kvar.

alltså när `DrawHangman(lives);` kommer:

```
 static void DrawHangman(int lives)
    {
        string[] stages =
        {
// 0
        @"
  +---+
  |   |
  O   |
 /|\  |
 / \  |
      |
=========",
```

osv… för varje antal liv.

Klassen ScoreData har två ints:

```
class ScoreData
{
    public int Wins { get; set; } = 0;
    public int Losses { get; set; } = 0;
}
```

Efter varje vinst eller förlust anropas SaveScore(bool won) som läser in den befintliga statistiken och uppdaterar den.

För varje inmatning kontrolleras självklart om gissningen inte är tom, om det är en bokstav och om bokstaven redan har gissats.

5 Resultat

En konsolapplikation för C# .NET har skapats och uppfyller alla ställda krav. appen är ett hänga gubbe spel med tydlig feedback. ASCII grafik som visar användaren hur nära hen är att förlora. Ett highscore som sparas mellan körningar. En felhantering av inmatning och en kod som är kommenterad.

Jag har har testat så att gissning av korrekt bosktav gör att bokstaven fylls i. Gissning av fel bokstav gör att livet minskar och ASCII figuren uppdateras. Gissning av samma bokstav flera gånger ger felmeddelande. En tom inmatning eller fler tecken avvisas. Vinst eller förlust uppdaterar highscoren och vid omstart finns highscore kvar.

6 Slutsatser

Projektuppgiften har gått förvånansvärt bra. Alla delar fungerar nu som tänkt och det har varit väldigt roligt att arbeta med C# .NET. Jag ser mig själv redan mer som en .NET utvecklare än en webbutvecklare och denna kurs är definitivt en av anledningarna varför. Det enda riktiga problemet jag stötte på under utvecklingen var bokstäverna i orden som gissades.

Om jag hade haft mer tid till arbetet och inte projektledningskursen hängandes över mig hade jag definitivt satsat mer på detta. Jag gjorde pga tidsbristen bara det absolut obligatoriska och inget mer.

Under uppgiftens gång har jag förbättrat mina kunskaper kring strängar/arrayer, en så kallad “game loop”, JSON och feedback till användaren.

Jag ser verkligen fram emot Webbutveckling med .NET kursen som kommer längre fram och vill tacka så mycket för denna kurs!

Källförteckning

[1] En rundtur i C#-språket https://learn.microsoft.com/sv-se/dotnet/csharp/tour-of-csharp/overview 2026. Hämtad 2026-10-20.

[2] JSON https://www.json.org/json-en.html 2026. Hämtad 2026-10-20.

[3] ASCII-Konst https://sv.wikipedia.org/wiki/ASCII-konst 2026. Hämtad 2026-10-21

[4] Git https://git-scm.com/about 2026. Hämtad 2026-10-22